

Activity 1 - Turn a specification into a Rust function

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K1/A1

Specification

line_total(unit_price_cents: u64, qty: u32) -> Option<u64>. qty 0 or above MAX_QTY (10,000) returns None; an overflowing product returns None; otherwise Some(total cents).

Part 1: Learn Rust prompts

Explain it simply

I am learning Rust. Explain Cargo, variables, mutability, constants, integer types and functions that return Option to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

Walk me through the sample

Here is samples/01_variables_and_types.rs from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

Quiz me

Quiz me on Cargo, variables, mutability, constants, integer types and functions that return Option with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles variables, integer types and overflow with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

Part 2: Build with AI (vibe-coding prompts)

Plan before code

Act as a senior Rust reviewer. Specification: line_total(unit_price_cents: u64, qty: u32) -> Option<u64>. qty 0 or above MAX_QTY (10,000) returns None; an overflowing product returns None; otherwise Some(total cents). Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Turn a specification into a Rust function" in starter/src so that the supplied tests pass. Keep the u64/u32 types and the Option return; use checked arithmetic only. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change

expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

Part 3: Rust sample scripts (samples/ folder)

01_variables_and_types.rs

Run: `rustc --edition 2021 01_variables_and_types.rs && ./01_variables_and_types`

```
// Sample: variables, mutability, constants and integer types.
const MAX_QTY: u32 = 10_000;

fn main() {
    let unit_price_cents: u64 = 250; // immutable by default
    let mut qty: u32 = 4; // mut: this value will change
    println!("price = {unit_price_cents} cents, qty = {qty}");
    qty += 1;
    println!("after adding one: qty = {qty} (max {MAX_QTY})");
    let total = unit_price_cents * u64::from(qty);
    println!(
        "total = {total} cents = ${} {:.02}",
        total / 100,
        total % 100
    );
    let big: u64 = u64::MAX;
    println!("checked_mul(2) on u64::MAX = {:?}", big.checked_mul(2));
}
```

Expected output:

```
price = 250 cents, qty = 4
after adding one: qty = 5 (max 10000)
total = 1250 cents = $12.50
checked_mul(2) on u64::MAX = None
```

02_functions_and_option.rs

Run: `rustc --edition 2021 02_functions_and_option.rs && ./02_functions_and_option`

```
// Sample: a function whose return type says "this can be refused".
fn line_total(unit_price_cents: u64, qty: u32) -> Option<u64> {
    if qty == 0 || qty > 10_000 {
        return None;
    }
    unit_price_cents.checked_mul(u64::from(qty))
}

fn main() {
    for (price, qty) in [(250, 4), (250, 0), (1, 10_001), (u64::MAX, 2)] {
        match line_total(price, qty) {
            Some(total) => println!("{qty} x {price} = {total}"),
            None => println!("{qty} x {price} rejected"),
        }
    }
}
```

Expected output:

```
4 x 250 = 1000
0 x 250 rejected
10001 x 1 rejected
2 x 18446744073709551615 rejected
```

Learn more

Rust Book: Hello, Cargo!: <https://doc.rust-lang.org/book/ch01-03-hello-cargo.html>

Rust Book: Variables and Mutability:

<https://doc.rust-lang.org/book/ch03-01-variables-and-mutability.html>

Rust Book: Data Types: <https://doc.rust-lang.org/book/ch03-02-data-types.html>

W3Schools: Rust Variables: https://www.w3schools.com/rust/rust_variables.php

W3Schools: Rust Data Types: https://www.w3schools.com/rust/rust_data_types.php

Programiz: Variables and Mutability: <https://www.programiz.com/rust/variables-mutability>

Programiz: Cargo: <https://www.programiz.com/rust/cargo>

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>

W3Schools: Get Started with Rust: https://www.w3schools.com/rust/rust_getstarted.php